

UNDO AND REDO TEXT EDITOR

A Stack-Based Console Application in C++

Presented by: Akhil Pratap Singh | 12410802

Lovely Professional Univesity

Introduction



- A text editor lets users type, edit, and manage text.
- Mistakes while typing are common and need to be corrected.
- Most editors solve this with two features: Undo and Redo.
- This project builds a simple console-based editor in C++ to demonstrate exactly how these two features work internally.

Objective

1. Understand and apply the Stack data structure practically
2. Build a simple console-based text editor in C++
3. Implement Undo and Redo using two stacks
4. Strengthen understanding of LIFO behaviour
5. Practice writing clean, modular, commented C++ code

Problem Statement



Beginners often learn the Stack data structure only in theory, without seeing it used in a real, everyday problem.

There is a need for a small project that clearly shows how a stack can track previous versions of data - so that changes can be undone and redone - using an example every student can relate to: a text editor.

Stack Concept (LIFO)

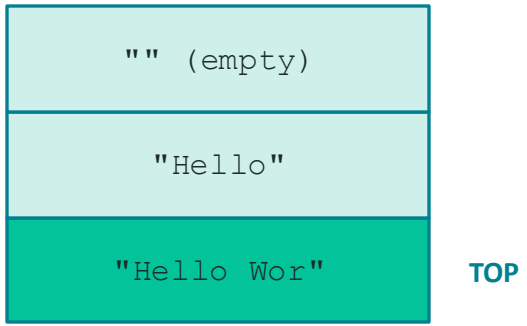
Stack = linear data structure following **Last In, First Out (LIFO)**.

Push
Insert an element on top

Pop
Remove the top element

Top / Peek
View top element only

undoStack (example)



Working of Undo and Redo



Undo Stack

Stores earlier versions of the text before each change





Redo Stack

Stores versions that were undone, in case user wants them back

Undo: push current text to Redo Stack -> restore top of Undo Stack
Redo: push current text to Undo Stack -> restore top of Redo Stack

Algorithm (Simplified)



1. Initialize empty text, empty Undo Stack and Redo Stack
2. Display menu and read user's choice
3. Add Text -> push old text to Undo Stack, then append new text
4. Undo -> push current to Redo Stack, restore top of Undo Stack
5. Redo -> push current to Undo Stack, restore top of Redo Stack
6. Repeat until user selects Exit

Source Code Overview



`class TextEditor`

`addText()`	Saves old text on Undo Stack, appends new text
`clearText()`	Saves old text on Undo Stack, empties current text
`undo()`	Restores previous state from Undo Stack
`redo()`	Restores undone state from Redo Stack
`display()`	Prints the current text to the console

main() uses a do-while loop with a switch statement to run the menu until Exit is chosen.

Sample Output



```
Enter your choice (1-6): 1
Enter text to add: Hello World
Text added successfully.

Enter your choice (1-6): 3
Undo successful.

Enter your choice (1-6): 2
Current Text: Hello

Enter your choice (1-6): 4
Redo successful.
```



Conclusion

This project successfully shows how two stacks can implement Undo and Redo in a text editor. It reinforces LIFO logic, STL stack usage, and modular C++ design through a simple, classroom-friendly example.

Thank You